NumPy Tutorial 5: Shape, Reshape, and Flatten

This chapter covers how to change the structure of arrays. After completing this tutorial, students will understand how to reshape arrays, flatten them, and understand why shape manipulation is important in Data Science.

---

1. Introduction

In previous tutorials, we learned how to create arrays and access their data. In this tutorial, we will learn how to change the shape and structure of arrays.

In real world Data Science and Machine Learning, data often needs to be in a specific shape before we can use it. For example, images in Machine Learning are stored as 3D arrays and need to be reshaped before processing.

---

2. Understanding Shape

Shape tells us the size of each dimension of an array.

```
import numpy as np

arr1 = np.array([1, 2, 3, 4, 5, 6])
arr2 = np.array([[1, 2, 3], [4, 5, 6]])

print(arr1.shape)
print(arr2.shape)
```

Output:

```
(6,)
(2, 3)
```

In this example:

- arr1 has shape (6,) — 6 elements, 1 dimension.
- arr2 has shape (2, 3) — 2 rows and 3 columns.

---

3. What is Reshape

Reshape means changing the shape of an array without changing its data.

The total number of elements must remain the same. For example:

- An array with 12 elements can be reshaped to (3, 4) or (4, 3) or (2, 6) or (2, 2, 3).
- But it cannot be reshaped to (3, 5) because that would require 15 elements.

---

4. **Reshaping 1D to 2D**

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

reshaped = arr.reshape(2, 3)
print(reshaped)
print("Shape:", reshaped.shape)
```

Output:

```
[[1 2 3]
 [4 5 6]]
Shape: (2, 3)
```

In this example:

- Original array had 6 elements in 1D.
- After reshape(2, 3) — it became 2 rows and 3 columns.
- Total elements = 2 × 3 = 6 — same as before.

---

5. **Reshaping to Different Shapes**

```
import numpy as np

arr = np.arange(1, 13)
print("Original:", arr)

shape1 = arr.reshape(3, 4)
print("3x4:\n", shape1)

shape2 = arr.reshape(4, 3)
print("4x3:\n", shape2)
```

```
shape3 = arr.reshape(2, 2, 3)
print("2x2x3:\n", shape3)
```

Output:

```
Original: [ 1  2  3  4  5  6  7  8  9 10 11 12]
3x4:
 [[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]
4x3:
 [[ 1  2  3]
  [ 4  5  6]
  [ 7  8  9]
  [10 11 12]]
2x2x3:
 [[[ 1  2  3]
   [ 4  5  6]]
  [[ 7  8  9]
   [10 11 12]]]
```

All three reshapes have the same 12 elements — just arranged differently.

---

6. **Using -1 in Reshape**

When we use -1 in reshape, NumPy automatically calculates that dimension for us.

```
arr = np.arange(1, 13)

reshaped = arr.reshape(3, -1)
print(reshaped)
print("Shape:", reshaped.shape)
```

Output:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape: (3, 4)
```

In this example:

- We said 3 rows and let NumPy figure out the columns.
- 12 elements ÷ 3 rows = 4 columns — NumPy calculated this automatically.

Using -1 is very common in Machine Learning code.

---

7. **What is Flatten**

Flatten converts any multi-dimensional array into a 1D array.

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

flat = arr.flatten()
print(flat)
print("Shape:", flat.shape)
```

Output:

```
[1 2 3 4 5 6 7 8 9]
Shape: (9,)
```

The 2D array with 3 rows and 3 columns became a single row of 9 elements.

---

8. **Flatten vs Ravel**

Both flatten and ravel convert an array to 1D. But there is one difference.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

flat = arr.flatten()
ravel = arr.ravel()

print(flat)
print(ravel)
```

Both give the same output:

```
[1 2 3 4 5 6]
[1 2 3 4 5 6]
```

The difference:

Method	Returns	Modifies Original
flatten()	A copy of the array	No
ravel()	A view of the array	Yes (if changed)

In most cases, flatten is safer to use for beginners.

---

9. **Transpose of an Array**

Transpose means swapping rows and columns. Rows become columns and columns become rows.

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print("Original shape:", arr.shape)
transposed = arr.T
print("Transposed shape:", transposed.shape)
print(transposed)
```

Output:

```
Original shape: (2, 3)
Transposed shape: (3, 2)
[[1 4]
 [2 5]
 [3 6]]
```

Transpose is used very frequently in linear algebra and Machine Learning.

---

10. **Adding New Dimensions**

Sometimes we need to add an extra dimension to an array. We use np.newaxis for this.

```
arr = np.array([1, 2, 3, 4, 5])
print("Original shape:", arr.shape)

arr_row = arr[np.newaxis, :]
print("Row vector shape:", arr_row.shape)
```

```
arr_col = arr[:, np.newaxis]
print("Column vector shape:", arr_col.shape)
```

Output:

```
Original shape: (5,)
Row vector shape: (1, 5)
Column vector shape: (5, 1)
```

This is commonly used when preparing data for Machine Learning models.

---

11. **Conclusion of Tutorial 5**

In this tutorial, students have learned:

1. What shape means in NumPy arrays.
2. How to reshape arrays from 1D to 2D and 3D.
3. How to use -1 in reshape for automatic dimension calculation.
4. How to flatten arrays using flatten and ravel.
5. The difference between flatten and ravel.
6. How to transpose an array using .T.
7. How to add new dimensions using np.newaxis.

In the next tutorial, we will learn about mathematical and statistical functions in NumPy.

This completes NumPy Tutorial 5.